

Uniwersytet Śląski

Sprawozdanie z projektu
Zbiory i Systemy Rozmyte

FuzzyVitals – System klasyfikacji obciążeń treningowych

Autor: Jakub Sikora
Nr albumu: 347903

Bytom, 2026

1. Wstęp

Niniejsze sprawozdanie opisuje projekt zaliczeniowy o nazwie FuzzyVitals, który został wykonany w ramach przedmiotu Zbiory i Systemy Rozmyte. Celem projektu było zaprojektowanie i implementacja systemu ekspertowego opartego na logice rozmytej, który byłby w stanie ocenić gotowość treningową użytkownika na podstawie kilku parametrów wejściowych.

Temat projektu wybrałem dlatego że uważam iż logika rozmyta dobrze nadaje się do modelowania takich sytuacji gdzie dane wejściowe są niepewne lub subiektywne, a dokładnie tak jest w przypadku oceny stanu fizycznego człowieka. Tradycyjne podejścia zero-jedynkowe nie oddają złożoności takich zjawisk – człowiek rzadko kiedy jest "w pełni wypoczęty" albo "całkowicie zmęczony", najczęściej jest gdzieś pomiędzy tymi stanami.

Projekt zrealizowałem samodzielnie, korzystając z języka Python oraz kilku bibliotek open-source. Interfejs użytkownika został zbudowany przy pomocy frameworku Streamlit, co pozwoliło na stworzenie prostej ale działającej aplikacji webowej bez potrzeby pisania kodu frontendowego.

2. Opis projektu i zastosowanej metody

FuzzyVitals to inteligentny system ekspertowy zbudowany w oparciu o model wnioskowania Mamdaniego. System przyjmuje na wejściu pięć parametrów opisujących stan fizyczny i psychiczny użytkownika, a następnie na ich podstawie wylicza sugerowaną intensywność treningu. Wynikiem jest przypisanie użytkownika do jednej z trzech klas: Regeneracja, Trening Lekki lub Trening Mocny.

Metoda Mamdaniego jest jedną z najpopularniejszych metod wnioskowania rozmytego. Polega ona na tym że zarówno poprzedniki jak i następni reguł są zbiorami rozmytymi. Wnioskowanie odbywa się w czterech krokach: fuzyfikacja wartości wejściowych, ewaluacja reguł, agregacja wyników cząstkowych i na koniec defuzyfikacja, czyli wyznaczenie wartości ostrej wyjścia. W projekcie zastosowałem metodę centroidu (Center of Gravity) jako metodę defuzyfikacji, ponieważ daje ona wygładzone i stabilne wyniki.

Kluczową zaletą takiego podejścia jest odporność systemu na sprzeczne dane wejściowe. Przykładowo, jeśli użytkownik zgłasza wysoką motywację ale równocześnie duże zmęczenie mięśniowe (DOMS) i słabą jakość snu, to system nie "zawiesza się" tylko poprawnie wyważa te informacje i wyznacza odpowiedni poziom intensywności. W systemach binarnych taka sytuacja byłaby trudna do obsłużenia.

3. Parametry wejściowe systemu

System analizuje pięć zmiennych wejściowych. Każda z nich została opisana funkcjami przynależności odpowiadającymi jej charakterystyce:

- Sen (regeneracja nocna)
- **Stres (obciążenie psychiczne)** – odzwierciedla subiektywny poziom stresu

użytkownika w skali 0–10. Wyższe wartości wskazują na większe obciążenie psychiczne co negatywnie wpływa na gotowość do wysiłku fizycznego.

- **DOMS (bolesność mięśniowa)** – Delayed Onset Muscle Soreness, czyli opóźniona bolesność mięśniowa po treningu. Parametr ten informuje system o stopniu regeneracji mięśni. Wysoki DOMS sugeruje że organizm jeszcze nie jest gotowy na kolejne intensywne obciążenie.
- **Motywacja (gotowość psychiczna)** – subiektywna ocena chęci do treningu. Motywacja jest moderatorem który może kompensować lub pogłębiać wpływ innych czynników na decyzję o intensywności.
- **Wiek (moderator wydolności)** – wpływa na progi aktywacji poszczególnych reguł. Starsze osoby mają generalnie dłuższy czas regeneracji, co system uwzględnia przy wyznaczaniu ostatecznej klasy wyjściowej.

Każdy z parametrów jest fuzzyfikowany, tzn. przypisywane są mu stopnie przynależności do odpowiednich zbiorów rozmytych. Dzięki temu wartość np. 6/10 dla snu nie jest twardą granicą między "złym" a "dobrym" snem, lecz może w pewnym stopniu należeć do obu zbiorów jednocześnie.

4. Zastosowane technologie

Projekt został zrealizowany w języku Python w wersji 3.9+. Wybór tego języka był podyktowany dostępnością biblioteki scikit-fuzzy, która dostarcza gotową implementację silnika logiki rozmytej zgodnego z modelem Mamdaniego. Poniżej omówię krótko każdą z użytych technologii:

- **scikit-fuzzy** – główny silnik obliczeniowy odpowiedzialny za definiowanie zbiorów rozmytych, funkcji przynależności oraz reguł bazy wiedzy. Biblioteka ta pozwala na wygodne deklaratywne opisanie całego systemu wnioskowania.
- **Streamlit** – framework do budowania interaktywnych aplikacji webowych w Pythonie. Dzięki niemu udało się stworzyć przyjazny interfejs graficzny z suwakami do wprowadzania danych wejściowych bez potrzeby znajomości HTML czy JavaScript.
- **Matplotlib** – biblioteka do wizualizacji danych. Zastosowałem ją do dynamicznego wyświetlania procesu defuzyfikacji, co pozwala użytkownikowi zobaczyć jak wyglądają aktywowane zbiory wyjściowe i gdzie wypada centroid.
- **NumPy** – używane wewnętrznie przez scikit-fuzzy do reprezentacji i przetwarzania universum zmiennych rozmytych jako tablic numerycznych.

Projekt zawiera automatyczny skrypt loader.bat który konfiguruje całe środowisko uruchomieniowe. Skrypt sprawdza wersję Pythona, instaluje wszystkie zależności a następnie uruchamia serwer Streamlit i otwiera aplikację w przeglądarce Chrome w trybie aplikacji (bez paska adresu i zakładek). Takie rozwiązanie znacznie upraszcza uruchomienie projektu przez osoby nieznające środowiska Pythona.

5. Działanie systemu i baza reguł

Serce systemu stanowi baza reguł rozmytych. Reguły zostały sformułowane w sposób zbliżony do języka naturalnego, np.: "JEŻELI sen jest dobry ORAZ stres jest niski ORAZ DOMS jest mały TO trening jest mocny". Zdefiniowałem kilkanaście takich reguł obejmujących różne kombinacje wartości wejściowych.

Proces wnioskowania przebiega następująco: po wprowadzeniu wartości przez użytkownika system wyznacza stopnie przynależności każdej zmiennej do odpowiednich zbiorów. Następnie dla każdej reguły wyliczana jest jej siła aktywacji (metodą minimum lub iloczynu algebraicznego). Aktywowane częściowo zbiory wyjściowe są agregowane metodą maksimum, a na końcu wyznaczany jest centroid całego wynikowego obszaru – stanowi on wartość ostrej zmiennej wyjściowej reprezentującej intensywność treningu.

Interfejs aplikacji pozwala użytkownikowi na obserwację całego tego procesu. W panelu bocznym ustawia się wartości suwaków, a w głównym obszarze wyświetlana jest rekomendacja wraz z wykresem defuzyfikacji. Można dzięki temu zobaczyć jak poszczególne reguły wpływają na kształt wynikowego obszaru rozmytego.

6. Instrukcja uruchomienia

Uruchomienie projektu zostało maksymalnie uproszczone dzięki skryptowi loader.bat. Jedynym wymaganiem jest posiadanie zainstalowanego Pythona w wersji 3.9 lub nowszej (dostępnego w zmiennej PATH systemu Windows).

Kroki uruchomienia:

- Upewnij się że Python 3.9+ jest zainstalowany i dostępny z poziomu wiersza poleceń.
- Kliknij dwukrotnie plik loader.bat.
- Skrypt automatycznie sprawdzi wersje Pythona, pobierze i zainstaluje wymagane biblioteki (streamlit, scikit-fuzzy, numpy, matplotlib).
- Po zakończeniu instalacji skrypt uruchomi serwer aplikacji i otworzy interfejs w przeglądarce Chrome.

W przypadku gdyby przeglądarka Chrome nie była zainstalowana aplikacja powinna otworzyć się w domyślnej przeglądarce systemowej. Interfejs jest dostępny pod adresem lokalnym (domyślnie localhost:8501).

7. Wnioski i podsumowanie

Projekt FuzzyVitals pozwolił mi na praktyczne zapoznanie się z implementacją systemu rozmytego od podstaw. Najtrudniejszym elementem okazało się dobieranie odpowiednich funkcji przynależności i kształtowanie bazy reguł tak, żeby wyniki systemu były intuicyjne i zgodne z tym czego można by się spodziewać na podstawie zdrowego rozsądku.

Zauważyłem że logika rozmyta bardzo dobrze sprawdza się w tego typu zastosowaniach – tam gdzie dane wejściowe są subiektywne i trudno je jednoznacznie sklasyfikować. System radzi sobie ze sprzecznymi danymi i nie produkuje gwałtownych skoków w wynikach przy małych zmianach wejścia, co jest dużą zaletą w porównaniu do klasycznych systemów decyzyjnych opartych na progach.

Pewnym ograniczeniem projektu jest to że reguły zostały zdefiniowane przeze mnie ręcznie, na podstawie intuicji a nie danych empirycznych. W bardziej zaawansowanej wersji można by zastosować metody automatycznego uczenia bazy reguł na podstawie

danych z rzeczywistych pomiarów. Myślę jednak że jak na projekt zaliczeniowy z przedmiotu Zbiory i Systemy Rozmyte, zrealizowany cel jest satysfakcjonujący i pokazuje możliwości tej technologii w praktycznym zastosowaniu.

Podsumowując: projekt zakończył się sukcesem. Aplikacja działa poprawnie, interfejs jest intuicyjny a wyniki wnioskowania rozmytego są spójne i sensowne. Zdobyta wiedza na pewno przyda mi się w przyszłości przy projektowaniu systemów sterowania lub wspomagania decyzji.

8. Literatura i źródła

- [1] Zadeh L.A., Fuzzy sets, Information and Control, 1965
- [2] Dokumentacja biblioteki scikit-fuzzy: <https://pythonhosted.org/scikit-fuzzy/>
- [3] Dokumentacja Streamlit: <https://docs.streamlit.io>
- [4] Mamdani E.H., Application of fuzzy algorithms for control of simple dynamic plant, 1974